

PROJECT

Finance Tracker

Authentication • CRUD APIs • Reports
Dashboard Analytics • File Uploads

Node.js • Express • MongoDB • JWT

MY FINANCE

Dashboard

Income

\$5,000

Expenses

\$1,450

Balance

\$3,550

RECENT ACTIVITY

Salary

+ 2,500

Rent

- 800

Food

- 250

What the backend provides

The backend manages users, transactions, reports and dashboard data through REST APIs.

CORE GOAL

- Personal Finance Tracker backend
- Manage users
- Manage income & expenses
- Generate reports
- Provide dashboard data

TECH STACK

- Node.js
- Express
- MongoDB + Mongoose
- JWT + bcryptjs
- Multer + xlsx

THE 4 BACKEND PARTS

- 01 Setup & Auth — Server, DB, Register, Login, JWT
- 02 Income CRUD — Add, Get, Delete, Excel
- 03 Expense CRUD — Add, Get, Delete, Excel
- 04 Dashboard & Uploads — Aggregation, Stats, Images

Client / Postman



Express Route



Middleware



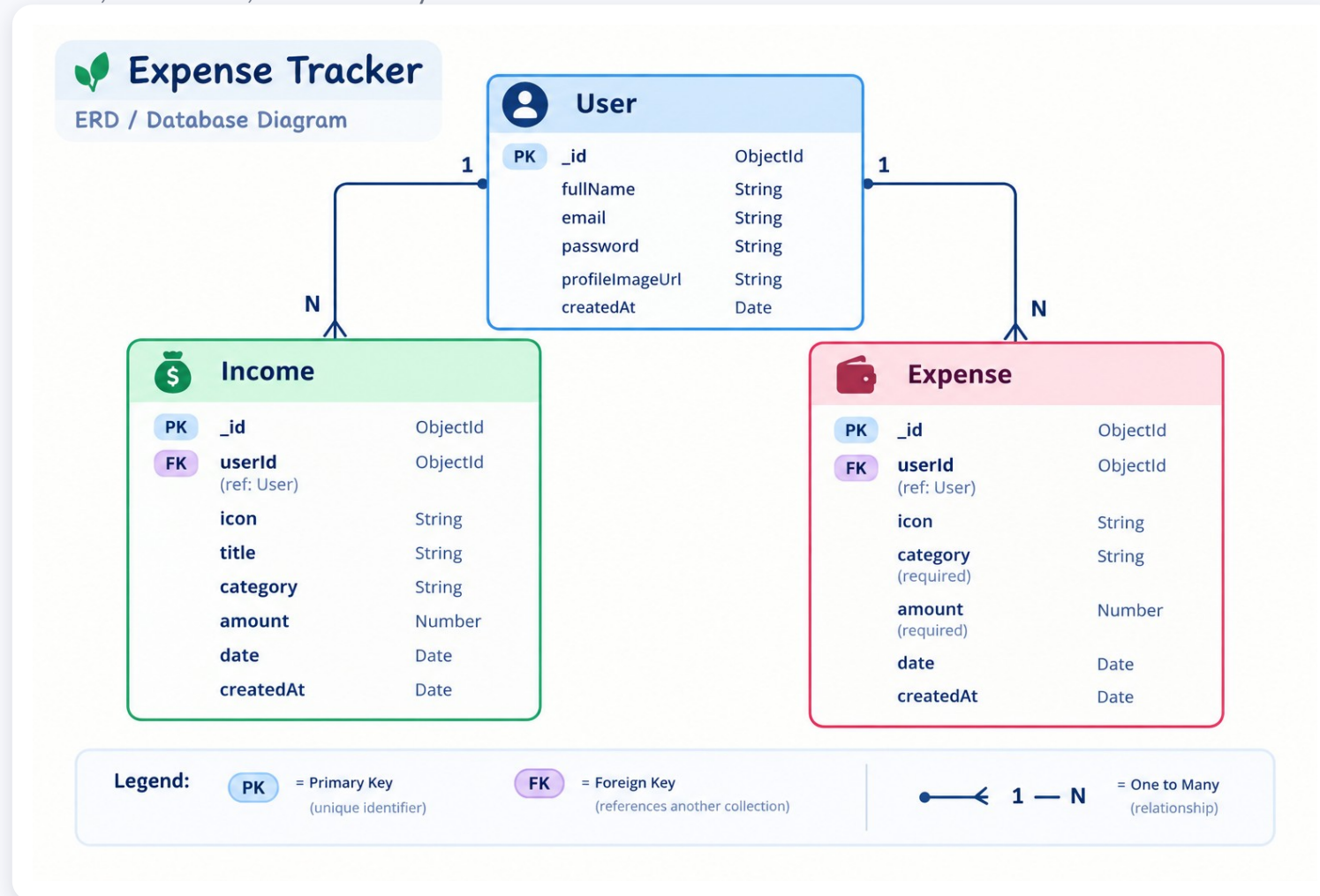
Controller



MongoDB

Entity Relationship Diagram

User, Income, and Expense collections, their fields, and how they relate.



1. Server setup + database + User model

Foundation: start the server, connect MongoDB, define users, and hash passwords before saving.

WHAT TO EXPLAIN

- Express server is created with CORS and JSON middleware
- Environment variables are loaded with dotenv
- Route modules for auth, income, expense and dashboard are mounted
- Server starts listening once setup completes

DATABASE & USER MODEL

- Mongoose connects to MongoDB using the connection string
- User schema defines email, password and profile fields
- A pre-save hook hashes the password with bcrypt before storing it
- A comparePassword method verifies credentials at login

Server



MongoDB



User Schema



bcrypt hash



Save User

2. Register → Login → JWT protection

Once authenticated, the JWT becomes the security key used by the protected APIs.

WHAT TO EXPLAIN

- Register creates the account
- bcryptjs stores a hashed password
- Login verifies the password
- JWT carries the user identity
- protect runs before protected controllers
- req.user becomes available to controllers

ROUTE PROTECTION

- protect middleware reads the Bearer token from request headers
- jwt.verify decodes the token and checks the secret
- The matching user is fetched and attached as req.user
- Requests without a valid token receive a 401 response

Register



Login



JWT



protect



req.user



Controller

1. Add + Get income

The income controller uses the authenticated user ID as the owner of every transaction.

WHAT TO EXPLAIN

- req.body carries icon, title, category, amount and date
- req.user.id identifies the owner
- Validation checks required fields
- new Income(...) creates the document
- save() writes it to MongoDB
- GET uses find({ userId })

VALIDATION RULES

- title, amount, and date are required fields
- icon and category are optional metadata
- Records are always scoped to the authenticated user
- GET returns only the current user's income list

POST /add



req.body



userId



save()



GET /get



find()

2. Delete + Excel report

CRUD is completed with secure deletion, then the same user data can be exported as an .xlsx file.

KEY POINTS

- `findOneAndDelete({ _id, userId })` checks both ID and owner
- If no record exists → 404
- Excel endpoint gets the user's records
- xlsx creates the workbook and the server sends it

EXPORT DETAILS

- Source, Category, Amount and Date columns are included
- Records are sorted by date, newest first
- Workbook is generated in memory with the xlsx library
- File streams back as a downloadable .xlsx

POST /add



GET /get



DELETE /:id



GET /downloadexcel

1. Add + Get expense

Expense follows the same request pattern as Income, but uses the Expense model and its own fields.

WHAT CHANGES?

- Fields: icon, category, amount, date
- category, amount and date are required
- req.user.id is stored as userId
- save() writes the expense
- GET uses find({ userId })
- Ownership rule stays the same

VALIDATION RULES

- category, amount, and date are required fields
- icon is optional metadata for the UI
- Records are always scoped to the authenticated user
- GET returns only the current user's expense list

POST /add



req.body



userId



save()



GET /get



find()

2. Delete + Excel report

The Expense module completes the same CRUD/report pattern used by Income.

PATTERN TO REMEMBER

- Route receives the transaction ID
- JWT middleware identifies the user
- Controller checks ID + userId
- Mongoose deletes the matching document
- Excel export uses the user's expense records

EXPORT DETAILS

- Category, Amount and Date columns are included
- Records are sorted by date, newest first
- Same workbook-generation pattern as Income
- File streams back as a downloadable .xlsx

POST /add



GET /get



DELETE /:id



GET /downloadexcel

1. Dashboard analytics + 30/60-day data

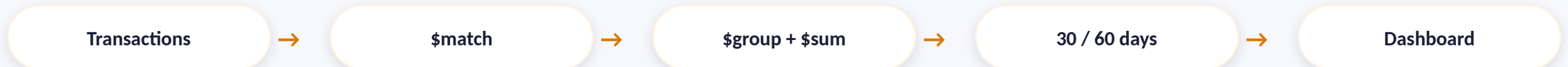
The dashboard calculates totals from MongoDB and retrieves recent transaction windows for the frontend.

WHAT TO EXPLAIN

- `$match` filters by the current user
- `$group` + `$sum` calculate total income
- The same aggregation calculates total expenses
- Last 30 days tracks expense transactions
- Last 60 days tracks income transactions

AGGREGATION PIPELINE

- `$match` filters documents to the logged-in user
- `$group` with `$sum` totals the amount field
- Same pipeline runs for both income and expense
- Recent-activity windows use a date range query



2. Profile image upload with Multer

Uploads are separate from transaction CRUD: the client sends a file, Multer validates it, and the backend stores its path.

UPLOAD FLOW

- Client sends multipart/form-data
- Multer uses diskStorage
- Filename gets a timestamp
- Allowed image MIME types are checked
- Saved file path is used by the profile image endpoint

VALIDATION RULES

- Only .jpeg, .jpg, and .png files are accepted
- Rejected files return a descriptive error message
- Filenames are prefixed with a timestamp to avoid collisions
- Saved path is returned to the client as imageUrl

Form-data



Multer



Type check



Save file



imageUrl

How the four parts connect

Final discussion: one protected request from Postman / frontend to MongoDB and back.



ROUTES

- /api/v1/auth/register
- /api/v1/auth/login
- /api/v1/income/*
- /api/v1/expense/*
- /api/v1/dashboard/
- /api/v1/auth/upload-image

POSTMAN CHECK

- Register → receive JWT
- Login → receive JWT
- Send Bearer token to protected routes
- Create income / expense
- Read dashboard totals
- Upload image → receive imageUrl

MAIN IDEA • Request → Middleware → Controller → Database → Response

MERN Finance Tracker Frontend

React client for the Finance Tracker · Auth screens, Dashboard analytics, Income & Expense management

React

Vite

Bootstrap

Axios

Recharts

React Router

React.js · Bootstrap · Axios · Recharts

MY FINANCE

Dashboard

Income

\$5,000

Expenses

\$1,450

Balance

\$3,550

RECENT ACTIVITY

Salary

+ \$2,500

Rent

- \$900

Freelance

+ \$600

What the frontend provides

The frontend gives users a fast, single-page interface to sign in, track spending, and visualize their finances against the REST API.

CORE GOAL

- Personal Finance Tracker client
- Authenticate users
- Visualize income & expenses
- Manage transactions in real time
- Export reports to Excel

TECH STACK

- React 18 + Vite
- Bootstrap 5
- React Router DOM
- Axios (with JWT interceptor)
- Recharts + Lucide icons

THE 4 FRONTEND PARTS

- 1 · Auth + Setup
- 2 · Dashboard
- 3 · Income Page
- 4 · Expense Page + API Setup

1. React setup + Bootstrap + routes

Person 1 lays the foundation every other screen builds on: the app shell, styling system, and the routes that guard private pages.

What to explain

- Vite scaffolds a fast React dev environment
- Bootstrap classes + custom CSS style the UI
- Routes split public vs. protected pages

1 Project Setup

- React app scaffolded with Vite for fast local development
- Bootstrap installed globally alongside a small custom stylesheet
- Environment variable stores the API base URL
- Dependencies installed: axios, react-router-dom, recharts, react-hot-toast

2 App Routes

- Public routes: Login and Sign Up screens
- Protected routes: Dashboard, Income, Expense
- A Private Route wrapper checks for a valid session
- Unauthenticated visitors are redirected to Login

2. Login UI, Sign Up UI + API calls

Once the forms are wired to the auth endpoints, the JWT returned from the API becomes the key that unlocks every protected screen.

What to explain

- Client-side validation before calling the API
- Profile Photo Selector attaches an avatar
- Token + user saved to localStorage on success

1 Login & Sign Up Screens

- Email and password fields with inline validation
- Clear error messages when a field is missing or invalid
- Profile Photo Selector lets a new user pick an avatar
- A loading state disables the button while submitting

2 Connecting to the API

- Form submit calls the login or sign-up endpoint
- The returned token and user info are saved locally
- On success, the user is redirected to the Dashboard
- On failure, the API's error message is shown to the user

1. Layout, summary cards + recent activity

The dashboard is the biggest surface area in the app — a persistent layout wrapping cards, lists, and charts fed by live API data.

What to explain

- Shared layout renders sidebar + navbar once
- One reusable Stat Card is shared for balance/income/expense
- Recent Transactions merges income + expense feeds

1 Summary Cards

- One reusable Stat Card component for balance, income, and expense
- Each card picks its own icon and color by type
- Values update live from the API without a page reload

2 Recent Transactions

- Income and expense records are merged into one feed
- Sorted by date, newest first
- Only the latest five entries are shown on the dashboard

2. Financial Overview: pie + bar charts

Recharts turns raw income/expense arrays into a pie split and a trend line, giving users an instant read on their finances.

What to explain

- Financial Overview: pie of income vs. expense
- Expense Trend: bar chart of spend over time
- Both re-render on new transactions, no page reload

1 Financial Overview

- Pie chart comparing total income against total expense
- Gives an instant visual read on overall balance
- Colors match the income/expense theme used across the app

2 Expense Trend

- Bar chart of spending over the last 30 to 60 days
- Helps spot spikes or unusual spending patterns
- Recalculates automatically as new expenses are added

1. Income overview + Add Income form

The Income page mirrors the Expense page's pattern: an overview chart on top, a controlled form beneath it to add new entries.

What to explain

- Income Overview chart summarizes income by source
- Add Income is a simple controlled form
- Submitting refreshes the list right away

1 Income Overview

- Chart summarizing income grouped by source
- Gives a quick read on where money is coming from
- Updates automatically as new income is added

2 Add Income Form

- Controlled form for source, amount, and date
- Shows a success toast once saved
- Refreshes the income list right after submitting

2. Income list, delete + download Excel

The list renders every income record with a delete action, and a single button streams the same data out as an Excel file.

What to explain

- Each income entry renders as a card with a delete action
- Deleting calls the API then updates the list instantly
- Download hits the Excel export endpoint directly

1 Income List

- Every income record shown as its own card
- Delete icon removes an entry with one click
- List updates immediately, no page refresh needed

2 Download Excel

- One button exports all income records
- Streams a ready-to-open .xlsx file to the browser
- Mirrors the same export pattern used on the Expense page

1. API paths + Axios instance

Person 4 builds the shared plumbing first: one Axios instance every feature module imports, so auth headers only get written once.

What to explain

- The base URL points at the API root once, centrally
- Request interceptor attaches the JWT automatically
- Response interceptor logs the user out on session expiry

1 One Shared Axios Client

- A single instance points at the API's base URL
- Every feature module (auth, income, expense) imports it
- Keeps configuration — timeouts, headers — in one place

2 Automatic Auth Handling

- The saved JWT is attached to every outgoing request
- A logged-out or expired session returns a 401
- On 401, the app clears the session and returns to Login

2. Expense overview, form, list + export

The Expense page reuses the same shape as Income — overview, add form, list — proving out the shared Axios setup end to end.

What to explain

- Add Expense Form captures category, amount, and date
- Expense List renders and deletes each record
- Download Excel mirrors the Income export flow

1 Expense Overview & Add Form

- Overview chart summarizes spending by category
- Add Expense form captures category, amount, and date
- Same controlled-form pattern as the Income page

2 Expense List & Export

- Every expense shown as its own card with a delete action
- List updates instantly after adding or removing an entry
- Download Excel button exports all expenses to a file

How the four parts connect

Final discussion: one authenticated request flows from a form all the way to a chart on screen.

